OR – Vaja 11 – DMA in USART oz. DBGU RS232

1. **Prezentacija OR_DMA_PDC_razsirjen.ppt**
2. **Reševanje nalog**
   - **Razložimo:**
     i. **delovanje PDC krmilnika, DMA prenosa**
     ii. **pomen registrov (kaj naredimo, da deluje)**
     iii. SFR pogled registrov
   - Dopolnitev projekta **Morsejevega oddajnika** – **sprejem niza znakov**

11. naloga: Neposredni dostop do pomnilnika (DMA)

Enoto USART3 oz- DBGU uporabljajte tako, da bo **nize znakov sprejemala in oddajala s pomočjo DMA prenosov.** Osnovne nastavitve enot USART3 oz. DBGU ostanejo enake kot pri nalogi z USART oziroma DBGU (podprograma _INIT). Pri tem dodatno napišite naslednje podprograme:

- **Podprogram RCV_DMA**, ki nastavi DMA krmilnik tako, da je pripravljen na **sprejem naslednjih N znakov.** Naslov polja, v katerega naj se znaki zapisujejo, naj dobi podprogram v registru R0, vrednost parametra N pa naj se prenese v podprogram preko registra R1. Parameter N ima lahko vrednost med 1 in 80. Če je vrednost parametra večja oziroma manjša, jo omejite na navedeni interval.

- **Podprogram SND_DMA**, ki nastavi DMA krmilnik tako, da je pripravljen na oddajo znakov. **Naslov niza naj se prenese v podprogram preko registra R0, dolžina niza pa preko registra R1.**

- **Podprogram CHANGE**, ki **zamenja velike in male črke v nizu znakov**. **Naslov izvornega niza dobi podprogram v registru R0, naslov ponornega niza v registru R1, dolžino obeh nizov pa v registru R2.**

V glavnem programu po začetnih nastavitvah (xxx_INIT) pokličite podprogram **SND_DMA**, ki odda začetni niz znakov. Nato pokličite še **RCV_DMA**, kjer počakajte, da se DMA prenos zaključi. S klicem podprograma **CHANGE** zamenjajte velike in male črke v sprejetem nizu. Rezultat pošljite nazaj v PC s pomočjo SND_DMA.

Nalogo lahko rešite tudi na STM32H7 sistemu. Serijska povezava že deluje preko USB kabla in ni potrebno posebno povezovanje.
Dokumentacija je podana posebej.

FRI-SMS: Delo z lokalno testno prevezavo (Local Loopback, brez kabla):
V glavnem programu po začetnih nastavitvah (DEBUG_INIT) pokličite podprogram RCV_DMA in sprožite sprejem niza znakov. Nato pokličite podprogram RCV_DMA in sprožite oddajo niza znakov. Počakajte da se DMA oddaja niza znakov zaključi. Zaradi lokalne prevezave se bodo oddani znaki hkrati tudi sprejeli, zato preverite še, ali se je sprejem tudi zaključil (zastavica ENDRX). S klicem podprograma CHANGE zamenjajte velike in male črke v sprejetem nizu in jih vpišite v niz, ki se bo oddal v naslednji iteraciji.

```
//-----------------------------
// DMA related definitions
//-----------------------------

//  AHB2ENR register offset is 0xDC (for enabling SRAMs)
       .equ    RCC_AHB2ENR,  0xDC // RCC AHB2ENR peripheral clock reg

//  AHB1ENR register offset is 0xD8
       .equ    RCC_AHB1ENR,  0xD8 // RCC AHB1ENR peripheral clock reg


// DMAMUX1 base address is 0x40020800
       .equ    DMAMUX1_BASE,  0x40020800 // DMAMUX1 base address

       .equ    DMAMUX1_C0CR,  0x00      // CR for Channel 0
       .equ    DMAMUX1_C1CR,  0x04      // CR for Channel 1

// DMA1 base address is 0x40020000
       .equ    DMA1_BASE,  0x40020000 // DMA1 base address
// DMA2 base address is 0x40020400
       .equ    DMA2_BASE,  0x40020400 // DMA2 base address

       .equ    DMA_USART3_TX_STREAM, 0    //Channel 0 on DMA1
       .equ    DMA_USART3_RX_STREAM, 1    //Channel 1 on DMA1



// DMA Registers definitions
       .equ    DMA_LISR, 0x00
       .equ    DMA_HISR, 0x04
       .equ    DMA_LIFCR, 0x08
       .equ    DMA_HIFCR, 0x0C

       .equ    DMA_SxCR_TX, 0x10 + 0x18 * DMA_USART3_TX_STREAM
       .equ    DMA_SxFCR_TX, 0x24 + 0x18 * DMA_USART3_TX_STREAM
       .equ   DMA_SxSNDTR_TX, 0x14 + 0x18 * DMA_USART3_TX_STREAM
       .equ    DMA_SxPAR_TX, 0x18 + 0x18 * DMA_USART3_TX_STREAM
       .equ   DMA_SxM0AR_TX, 0x1C + 0x18 * DMA_USART3_TX_STREAM


       .equ    DMA_SxCR_RX, 0x10 + 0x18 * DMA_USART3_RX_STREAM
       .equ    DMA_SxFCR_RX, 0x24 + 0x18 * DMA_USART3_RX_STREAM
       .equ   DMA_SxSNDTR_RX, 0x14 + 0x18 * DMA_USART3_RX_STREAM
       .equ    DMA_SxPAR_RX, 0x18 + 0x18 * DMA_USART3_RX_STREAM
       .equ   DMA_SxM0AR_RX, 0x1C + 0x18 * DMA_USART3_RX_STREAM

// Comment following line when FLASH Linker Script is used
#define RAM_LinkScript
```

```
// Start of data section
            .data

            .align

LEDSTAT: .word 0 // LED state
MSECCNT: .word 0 //MSecs counter for SysTick_Handler
MSECMAX: .word 500 //MSecs interval for SysTick_Handler



#ifdef RAM_LinkScript
// Start of sram section
            .section .sram,"a",%progbits
#endif

            .align
NIZ1:  .space 12

            .align
NIZ2:  .asciz "Testni niz!"  // 12 bytes

            .equ    NIZ_LEN, 12
```

```
main:

#ifdef RAM_LinkScript

// Relocating Vector table to RAM (only for RAM Linker Script)
        bl RELLOC_VECTBL

/* Enable SRAMs and copy initial values only with RAM Linker script */
//   Enable SRAMs internal memories (for DMA1)
        bl  SRAM_ENABLE

//   Copy initial values for .sram section (from .text to SRAM)
             ldr r0,=_sisram
             ldr r1,=_ssram
             ldr r2,=_esram
             bl  MEM_COPY

#endif

             bl INIT_IO

             bl INIT_TC     // Priprava SysTick časovnika

             bl INIT_USART3 // Priprava USART3 naprave

             bl INIT_DMA    // Priprava DMA naprave za prenose preko USART3

// Main loop for USART3+DMA Echo test for blocks of 12 characters with change to upper
case
loop:

    ldr r0,=NIZ2
    mov r1,#NIZ_LEN
    bl SND_DMA

    bl LED_ON      // Vklop LED diode

    mov r0,#500
//    bl DELAY // Zakasnitev SW Delay: r0 x 1msec
    bl DELAYTC // Zakasnitev SysTick : r0 x 1msec

    ldr r0,=NIZ1
    mov r1,#NIZ_LEN
    bl RCV_DMA

    ldr r0,=NIZ1
    ldr r1,=NIZ2
    ldr r2, =NIZ_LEN
    bl CHANGE

    bl LED_OFF      // Izlop LED diode

    mov r0,#500
//    bl DELAY // Zakasnitev SW Delay: r0 x 1msec
    bl DELAYTC // Zakasnitev SysTick : r0 x 1msec

    b loop          // skok na vrstico loop:

__end:   b   __end
```

```
SRAM_ENABLE:
  push {r5, r6, lr}

       // Enable internal SRAMs (bits 31,30,29 in AHBeENR register)
       ldr r6, =RCC_BASE          // Load peripheral clock reg base address to r6
       ldr r5, [r6,#RCC_AHB2ENR]  // Read its content to r5
       orr r5, #(0b111 << 29)     // Set bits 31,30,29 to 1 to enable SRAMs clock
       str r5, [r6,#RCC_AHB2ENR]  // Store result in peripheral clock register

  pop  {r5, r6, pc}



MEM_COPY:
  push {r3, lr}

copy:
  ldr r3,[r0],#4
  str r3,[r1],#4

  cmp r1,r2
  blo copy

  pop  {r3, pc}



CHANGE:
    push {r3-r4,lr}

ch_zanka:
    ldrb r4, [r0], #1
    bic r3, r4, #0b100000   // zbrisi b5

    cmp r3, #'A'
    blo pisi

    cmp r3, #'Z'
    bhi pisi

    eor r4, r4, #0b100000   // spremeni crko

pisi:
    strb r4, [r1], #1  /* shranimo v niz2*/

    subs r2, r2, #1
    bne ch_zanka

    pop {r3-r4,pc}
```

```
INIT_DMA:
  push {r5, r6, lr}

    //-----------------------------------------
      // ------------- DMA1 Settings

      // Enable DMA1 Peripheral Clock (bit 0 in AHB1ENR register)
      ldr r6, =RCC_BASE          // Load peripheral clock reg base address to r6
      ldr r5, [r6,#RCC_AHB1ENR]  // Read its content to r5
      orr r5, #1              // Set bit 0 to enable DMA1 clock
      str r5, [r6,#RCC_AHB1ENR]  // Store result in peripheral clock register

      ldr r6, =DMA1_BASE          // Load DMA1 BASE address to r6

      // ------- DMA1 TX Settings
      // Disable DMA TX Channel
      ldr r5, [r6,#DMA_SxCR_TX]  // Read its content to r5
      bic r5, #1
      str r5, [r6,#DMA_SxCR_TX]    // Store result

wt0_EN0:
      ldr r5, [r6,#DMA_SxCR_TX]  // wait until bit is read as 0
      tst r5, #1
      bne wt0_EN0


      // Set MINC (increment memory pointer) and Direction (Memory->Peripheral)
      orr r5,r5,#(0b10001 << 6)    // b10=1 and bit7,6 = 01
      str r5, [r6,#DMA_SxCR_TX]    // Store result

      // Enable direct mode
      ldr r5, [r6,#DMA_SxFCR_TX]  // Read its content to r5
      bic r5, #0b100
      str r5, [r6,#DMA_SxFCR_TX]    // Store result

      // ------- DMA1 RX Settings
        // Disable DMA RX Channel
      ldr r5, [r6,#DMA_SxCR_RX]  // Read its content to r5
      bic r5, #1
      str r5, [r6,#DMA_SxCR_RX]    // Store result

wt1_EN0:
      ldr r5, [r6,#DMA_SxCR_RX]  // wait until bit is read as 0
      tst r5, #1
      bne wt1_EN0

      // Set MINC (increment memory pointer) and Direction (Memory<-Peripheral)
      orr r5,r5,#(0b10000 << 6)    // b10=1 and bit7,6 = 00
      str r5, [r6,#DMA_SxCR_RX]    // Store result

        // Enable direct mode
      ldr r5, [r6,#DMA_SxFCR_RX]  // Read its content to r5
      bic r5, #0b100
      str r5, [r6,#DMA_SxFCR_RX]    // Store result

//-----------------------------------------------
// ------------- DMAMUX1 Settings
// Set channels to devices translations (multiplexing)
```

```
      ldr r6, =DMAMUX1_BASE        // Load reg base address to r6

      mov r5,#46  // USART3_TX DMA Device Nr. is 46
    str r5, [r6, DMAMUX1_C0CR] // DMAREQ for Channel 0 to USART3_TX
      mov r5,#45  // USART3_Rx DMA Device Nr. is 45
    str r5, [r6, DMAMUX1_C1CR] // DMAREQ for Channel 1 to USART3_RX

//---------------------------------------------
// ------------- USART3 Settings
      ldr r6, =USART3_BASE      // Load USART3 BASE address to r1

// Disable USART3
      ldr r5, [r6,#USART_CR1]  // Read its content to r5
      bic r5, #1
      str r5, [r6,#USART_CR1]  // Store result

// Enable DMA Transmit and Receive for USART3
    ldr r5, [r6, #USART_CR3]
      orr r5, #(0b11<<6)          // Set bits 7 and 6 to enable DMAT and DMAR bits
      str r5, [r6,#USART_CR3]    // Store result

// Enable USART3
      ldr r6, =USART3_BASE      // Load USART3 BASE address to r6
      ldr r5, [r6,#USART_CR1]  // Read its content to r5
      orr r5, r5, #1
      str r5, [r6,#USART_CR1]  // Store result


      pop {r5, r6, pc}
```

```
RCV_DMA:
       push {r5, r6, lr}

       ldr r6, =DMA1_BASE          // Load reg base address to r6

WAIT_EN: // Wait EN bit to become zero
      ldr r5, [r6, #DMA_SxCR_RX]
      tst r5, #1
      bne WAIT_EN

       ldr r6, =DMA1_BASE          // Load reg base address to r6

//  Receive (RX) DMA Init
       ldr r5, =USART3_BASE+USART_RDR // RX peripheral address to r5
       str r5, [r6,#DMA_SxPAR_RX]  // Store peripheral DMA pointer
       str r0, [r6,#DMA_SxM0AR_RX]  // Store address pointer
       str r1, [r6,#DMA_SxSNDTR_RX]  // Store number of units

// Clear flags in Status register
       mov r5,#(0b111101<<6)          // clear flags for Ch1 in ISR
       str r5, [r6,# DMA_LIFCR]      // Store

//  Enable DMA Channel
       ldr r5, [r6, #DMA_SxCR_RX]
       orr r5, r5, #1
    str r5, [r6, #DMA_SxCR_RX] // Enable channel

// Wait for the end of reception (TCIF
       ldr r6, =DMA1_BASE          // Load reg base address to r6

WAIT_RC:
    ldr r5, [r6, #DMA_LISR]
    tst r5, #(1 << 11)        // TCIF1 flag
    beq WAIT_RC

       pop {r5, r6, pc}
```

```
SND_DMA:
       push {r5, r6, lr}

       ldr r6, =DMA1_BASE          // Load reg base address to r6

WAIT_EN1: // Wait EN bit to become zero
    ldr r5, [r6, #DMA_SxCR_TX]
    tst r5, #1
    bne WAIT_EN1

       ldr r6, =DMA1_BASE          // Load reg base address to r6
       //  Transmit (TX) DMA Init
       ldr r5, =USART3_BASE+USART_TDR // RX peripheral address to r5
       str r5, [r6,#DMA_SxPAR_TX]  // Store result in peripheral DMA pointerclock
register
       str r0, [r6,#DMA_SxM0AR_TX]  // Store address pointer
       str r1, [r6,#DMA_SxSNDTR_TX]  // Store result in peripheral DMA pointerclock
register

       // Clear flags in Status register
       mov r5, #0b111101          // Clear all bits for channel 0
       str r5, [r6,#DMA_LIFCR]  // Store result in peripheral DMA pointerclock register

       //  Enable DMA Channel
       ldr r6, =DMA1_BASE          // Load reg base address to r6
       ldr r5, [r6, #DMA_SxCR_TX]
       orr r5, r5, #1
    str r5, [r6, #DMA_SxCR_TX] // Enable channel

    // Wait for the end of transmission
    ldr r6, =USART3_BASE

WAIT_TC:
       ldr r5, [r6, #USART_ISR]
    tst r5, #(1 << 6)              // Test TC bit
    beq WAIT_TC

       pop {r5, r6, pc}
```

```
INIT_USART3:
            push {r0, r1, r2, lr}

            // Enable USART3 Peripheral Clock (bit 18 in APB1LENR register)
            ldr r1, =RCC_BASE          // Load peripheral clock reg base address to r1
            ldr r0, [r1,#RCC_APB1LENR]  // Read its content to r0
            orr r0, r0, #(1<<18)          // Set bit 18 to enable USART3 clock
            str r0, [r1,#RCC_APB1LENR]  // Store result in peripheral clock register

            // Enable GPIOB Peripheral Clock (bit 1 in AHB4ENR register)
            ldr r1, = RCC_AHB4ENR      // Load peripheral clock reg address to r6
            ldr r0, [r1]                // Read its content to r5
            orr r0, r0, #0b10              // Set bit 1 to enable GPIOB clock
            str r0, [r1]                // Store result in peripheral clock register

            ldr r1, =GPIOB_BASE          // Load GPIOB BASE address to r1

            // Make GPIOB Pina 10,11 as AF (bits 20:23 in MODER register)
            ldr r0, [r1,#GPIOx_MODER]  // Read GPIO_MODER content to r0
            ldr r2, =0xFF0FFFFF          // Clear mask
            and r0, r0, r2                 // Clear bits
            orr r0, #0x00A00000          // Write 10 to bits
            str r0, [r1,#GPIOx_MODER]    // Store result in GPIO MODER register

            // Make GPIOB Pina 10,11 as AF7 (bits 8:15 in AFRH register)
            ldr r0, [r1,#GPIOx_AFRH]  // Read GPIOB AFRH content to r0
            ldr r2, =0xFFFF00FF          // Clear mask
            and r0, r0, r2              // Clear bits
            orr r0, r0, #GPIO_AFRH_VALUE
            str r0, [r1,#GPIOx_AFRH]  // Store result in GPIOB AFRH register


            ldr r1, =USART3_BASE      // Load USART3 BASE address to r1

              // Disable USART3
            mov r0, #0
            str r0, [r1,#USART_CR1]               // Store result

            // Set USART3 BaudRate
            ldr r0, =USART_BRR_VAL
            str r0, [r1,#USART_BRR]               // Store result

            // Start USART3
            mov r0, #USART_CR1_VAL
            str r0, [r1,#USART_CR1]               // Store result

            pop {r0, r1, r2, pc}
```

```
RECV_UART:
             push {r1, r2, lr}

       ldr r1, =USART3_BASE
RECV_LP:
           ldr r2, [r1, #USART_ISR]
           tst r2, #(1 << 5)  // RXNE flag
           beq RECV_LP

           ldr r0, [r1, #USART_RDR]

             pop {r1, r2, pc}


SEND_UART:
             push {r1, r2, lr}

             ldr r1, =USART3_BASE
SEND_LP:
         ldr r2, [r1, #USART_ISR]
           tst r2, #(1 << 7)  // TXE flag
           beq SEND_LP

           str r0, [r1, #USART_TDR]

           pop {r1, r2, pc}


INIT_TC:
             push {r0, r1, lr}
             ldr r1, =SCS_BASE

             ldr r0, =SYSTICK_RELOAD_1MS
             str r0, [r1, #SCS_SYST_RVR]

             mov r0, #0
             str r0, [r1, #SCS_SYST_CVR]

             mov r0, #0b101
             str r0, [r1, #SCS_SYST_CSR]

             pop {r0, r1, pc}
```

```
INIT_IO:
            push {r5,r6,lr}


            // 1. RCC_AHB4ENR(Peripheral Clock Register): b8=1 .. Port I Enable
            // Enable GPIOI Peripheral Clock (bit 8 in AHB4ENR register)
            ldr r6, = RCC_AHB4ENR       // Load peripheral clock reg address to r6
            ldr r5, [r6]                // Read its content to r5
            orr r5, #0x00000100          // Set bit 8 to enable GPIOI clock
            str r5, [r6]                // Store result in peripheral clock register

            // 2. MODER (Mode Register): 01: General purpose output mode
            // Make GPIOI Pin13 as output pin (bits 27:26 in MODER register)
            ldr r6, =GPIOI_BASE       // Load GPIOI BASE address to r6
            ldr r5, [r6,#GPIOx_MODER]  // Read GPIOI_MODER content to r5
            and r5, #0xF3FFFFFF          // Clear bits 27-26 for P13
            orr r5, #0x04000000          // Write 01 to bits 27-26 for P13
            str r5, [r6]                // Store result in GPIO MODER register

            pop {r5,r6,pc}


// Delay with internal timer based loop approx. r0 x ms
DELAYTC:
          push {r1, r2, lr}
          ldr r1, =SCS_BASE

LOOPTC:             ldr r2, [r1, #SCS_SYST_CSR]
                    tst r2, #0x10000   // COUNT_FLAG=1?
                    beq LOOPTC

      subs r0, r0, #1
      bne LOOPTC

      pop {r1, r2, pc}

LED_ON:
            push {r5,r6,lr}

            ldr r6, =GPIOI_BASE       // Load GPIOI BASE address to r6
            // Led On
            mov    r5, #LEDs_ON
            str    r5, [r6,#GPIOx_BSRR] // Write to BSRR register

            pop {r5,r6,pc}


LED_OFF:
            push {r5,r6,lr}

            ldr r6, =GPIOI_BASE       // Load GPIOI BASE address to r6
            // Led Off
            mov    r5, #LEDs_OFF
            str    r5, [r6,#GPIOx_BSRR] // Write to BSRR register

            pop {r5,r6,pc}
```

11. naloga: FRISMS - neposredni dostop do pomnilnika (DMA)

Enoto DBGU uporabljajte tako, da bo **znake sprejemala in oddajala s pomočjo DMA prenosov**. Osnovne nastavitve ostanejo enake kot v 10. nalogi (podprogram DEBUG_INIT). Pri tem napišite naslednje podprograme:

- Podprogram **RCV_DMA**, ki nastavi **DMA krmilnik** tako, da je pripravljen na **sprejem naslednjih N znakov**. **Naslov polja**, v katerega naj se znaki zapisujejo, naj dobi podprogram **v registru R0**, vrednost **parametra N** pa naj se prenese v podprogram preko **registra R1**. Parameter N ima lahko vrednost **med 1 in 80**. Če je vrednost parametra večja oziroma manjša, **jo omejite** na navedeni interval.

- Podprogram **SND_DMA**, ki nastavi DMA krmilnik tako, da je pripravljen na **oddajo znakov**. **Naslov niza** naj se prenese v podprogram preko **registra R0**, **dolžina niza** pa preko **registra R1.**

- Podprogram **CHANGE**, ki **zamenja velike in male črke** v nizu znakov. **Naslov izvornega niza** dobi podprogram v **registru R0**, **naslov ponornega niza v registru R1**, **dolžino obeh** nizov pa v **registru R2**.

V **glavnem programu** po začetnih nastavitvah (**DEBUG_INIT**) pokličite podprogram **RCV_DMA**, nato pa v zanki čakajte, da se DMA prenos zaključi (**zastavica ENDRX**).

S klicem podprograma **CHANGE** zamenjajte velike in male črke v sprejetem nizu.

Rezultat pošljite s pomočjo **SND_DMA**.

```
  .equ DBGU_BASE, 0xFFFFF200 /* Debug Unit Base Address */
  .equ DBGU_CR, 0x00         /* DBGU Control Register */
  .equ DBGU_MR, 0x04         /* DBGU Mode Register*/
  .equ DBGU_IER, 0x08        /* DBGU Interrupt Enable Register*/
  .equ DBGU_IDR, 0x0C        /* DBGU Interrupt Disable Register */
  .equ DBGU_IMR, 0x10        /* DBGU Interrupt Mask Register */
  .equ DBGU_SR,  0x14        /* DBGU Status Register */
  .equ DBGU_RHR, 0x18        /* DBGU Receive Holding Register */
  .equ DBGU_THR, 0x1C        /* DBGU Transmit Holding Register */
  .equ DBGU_BRGR, 0x20       /* DBGU Baud Rate Generator
Register*/

  .equ STRING_LENGTH, 10

  niz1: .space STRING_LENGTH
  niz2: .space STRING_LENGTH
  .equ DBGU _RPR, 0x100 /* Receive Pointer Register */
  .equ DBGU _RCR, 0x104 /* Receive Counter Register */
  .equ DBGU _TPR, 0x108 /* Transmit Pointer Register */
  .equ DBGU _TCR, 0x10C /* Transmit Counter Register */
  .equ DBGU _RNPR, 0x110 /* Receive Next Pointer Register */
  .equ DBGU _RNCR, 0x114 /* Receive Next Counter Register */
  .equ DBGU _TNPR, 0x118 /* Transmit Next Pointer Register */
  .equ DBGU _TNCR, 0x11C /* Transmit Next Counter Register */
  .equ DBGU _PTCR, 0x120 /* Periph. Transfer Control Register */
  .equ DBGU _PTSR, 0x124 /* Periph. Transfer Status Register */
```

```
    bl DEBUG_INIT

zanka:
/* sproži sprejemanje preko DMA
*/
   ldr r0, =niz1
   ldr r1, =STRING_LENGTH
   bl RCV_DMA

   ldr r0,=DBGU_BASE
/* pocakaj na zastavico ENDRX */
z1: ldr r1, [r0, #DBGU_SR]
   tst r1, #1 << 3
   beq z1

/* zamenjaj velikost crk */
   ldr r0, =niz1
   ldr r1, =niz2
   ldr r2, =STRING_LENGTH
   bl CHANGE
/* sproži oddajanje preko DMA */
   ldr r0, =niz2
   ldr r1, =STRING_LENGTH
   bl SND_DMA

   b zanka
```

```
DEBUG_INIT:
   stmfd r13!, {r1, r14}
   ldr r0, =DBGU_BASE
   mov r1, #26
   str r1, [r0, #DBGU_BRGR]
   mov r1, #(1 << 11)
   str r1, [r0, #DBGU_MR]
   mov r1, #0x50
   str r1, [r0, #DBGU_CR]
   ldmfd r13!, {r1, pc}
```

```
RCV_DMA:
   stmfd r13!, {r2-r3,r14}
   ldr r2,=DBGU_BASE
   add r0,r0,#0x200000
   str r0,[r2,#DBGU_RPR]   /* kaz. niz1*/
   cmp r1, #1
   movlo r1, #1
   cmp r1, #80
   movhi r1, #80
   str r1,[r2,#DBGU_RCR]
   mov r3,#1          /* sprejem- RXTEN*/
   str r3,[r2,#DBGU_PTCR]

   ldmfd r13!, {r2-r3,pc}
```

```
SND_DMA:
   stmfd r13!, {r2-r3,r14}
   ldr r2,=DBGU_BASE
   add r0,r0,#0x200000
   str r0,[r2,#DBGU_TPR]   /* kaz. niz1*/
   str r1,[r2,#DBGU_TCR]   /*st. znakov*/
   mov r3,#1<<8        /*oddaja-TXTEN*/
   str r3,[r2,#DBGU_PTCR]

/* pocakaj na zastavico ENDTX */
zs: ldr r3, [r2, #DBGU_SR]
   tst r3, #1 << 4
   beq zs

   ldmfd r13!, {r2-r3,pc}
```

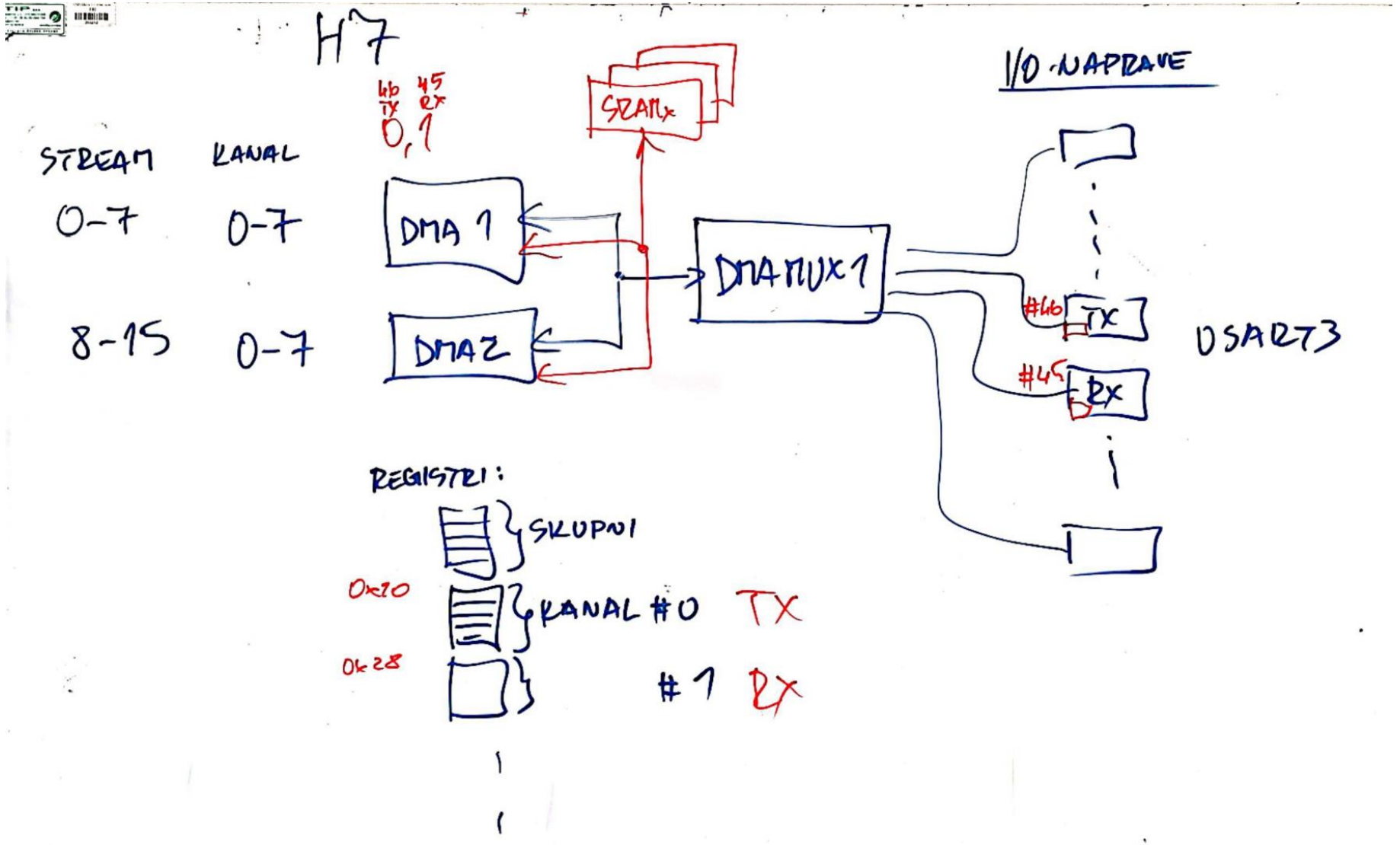
CHANGE:

```
CHANGE:
   stmed r13!, {r1-r4,r14}
ch_zanka:
   ldrb r4, [r0], #1

   bic r3, r4, #0b100000
@ v veliko črko (a=97, A=65)
```

```
   cmp r3, #'A' @ je črka ?  (med 'A' in 'Z')
   blo pisi
   cmp r3, #'Z'
   bhi pisi
   eor r4, r4, #0b100000    /*veliko crko v majhno*/
```

```
pisi:
   strb r4, [r1], #1
/* shranimo v niz2*/
   subs r2, r2, #1
   bne ch_zanka
   ldmed r13!, {r1-r4,pc}
```


H7
46 TX
45 RX
0, 1
STREAM
KANAL
0-7
0-7
8-15
0-7
SRAMx
DMA 1
DMA 2
DMAMUX1
I/O NAPRAVE
#46
TX
#45
RX
USART3
REGISTRI:
SKUPNI
0x20
KANAL #0 TX
0x28
#1 RX

DBGU RS232 + DMA (PDC)

Direct Memory Access

DMA PRENOSA IZ/V DBGU:

- **BRANJE NIZA ZNAKOV:**
  1. ZAČ. NASLOV → _RPR REG.
  2. ŠT. ZNAKOV → _RCR REG.
  3. ZAČNI PRENOS → RXTEN v _PTCR REG.

  KONEC? → ENDRX v DBGU_SR

- **PISANJE NIZA ZNAKOV:**
  1. ZAČ. NASLOV → _TPR REG.
  2. ŠT. ZNAKOV → _TCR REG.
  3. ZAČNI PRENOS → TXTEN v _PTCR REG.

  KONEC? → ENDTX v DBGU_SR

CPE (PROGRAM)	PDC (DMA)
ZAČ. NASLOV	ZAČ. NASLOV + 0x20000000

NIZ1: .SPACE 20

ZAMENJAVA VELIKE, MALE ČRKE

JE ČRKA?

DA, NEGIRAJ b5

NE, PUSTI